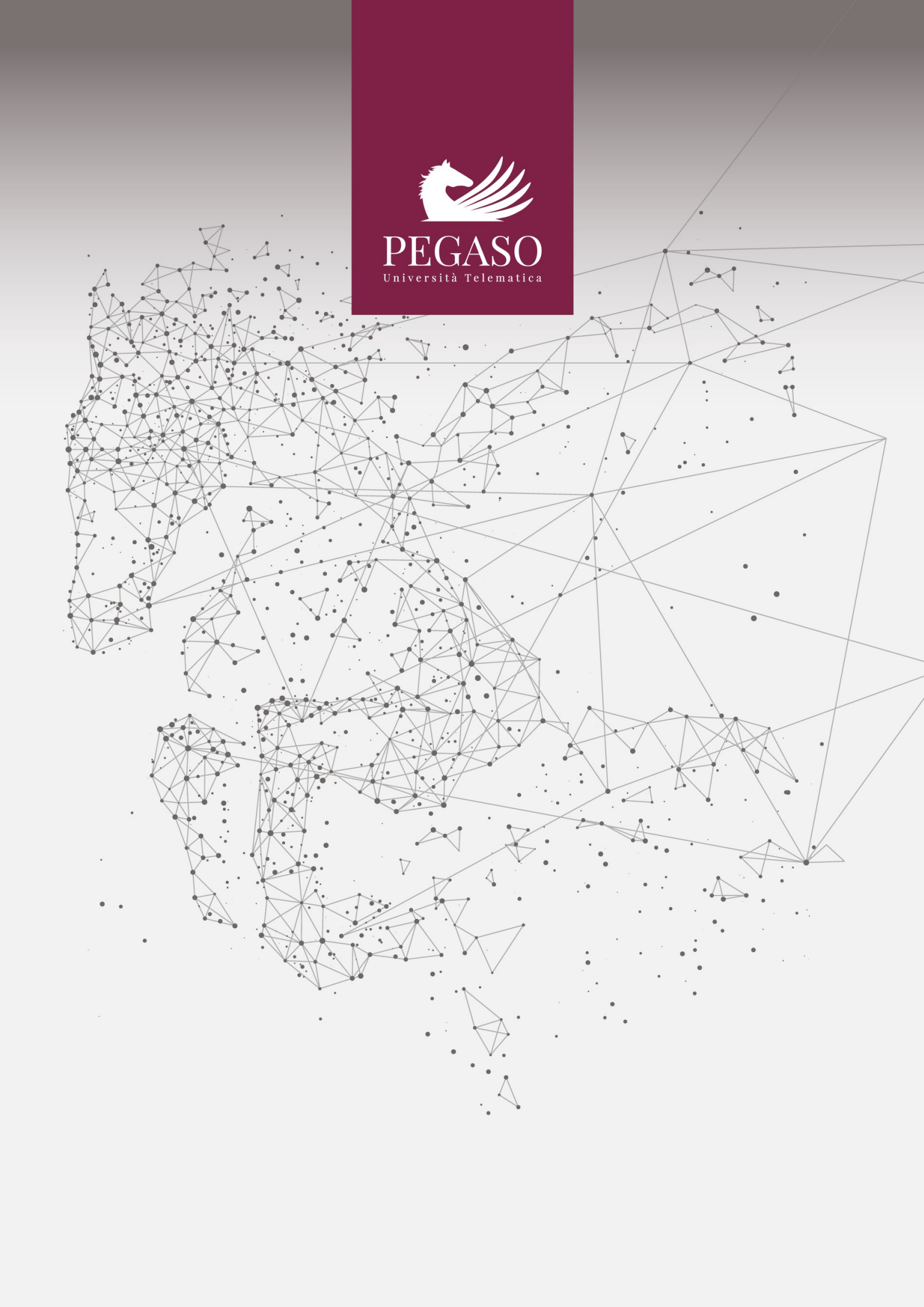

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. LISTA 3
2. OPERAZIONI SULLA LISTA 6
3. IMPLEMENTAZIONE CON SMART POINTERS 11
4. ANALISI DELLA COMPLESSITÀ 16
BIBLIOGRAFIA 17

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Lista

Si dice lista una tripla $L = (E; t; S)$ dove E è un insieme di elementi, $t \in E$ è detto testa ed S è una relazione binaria su E, cioè $S \subseteq E \times E$ che soddisfa le seguenti proprietà:

- Per ogni $e \in E, (e,t) \notin S$
- Per ogni $e \in E$, se $e \neq t$ allora esiste uno ed un solo $e' \in E$ tale che $(e',e) \in S$.
- Per ogni $e \in E$, esiste al più un solo $e' \in E$ tale che $(e',e) \in S$.
- Per ogni $e \in E$, se $e \neq t$ allora e è raggiungibile da t, cioè esistono $e'_1, \dots, e'_k \in E$ con $k \geq 2$ tali che $e'_1 = t, (e'_i, e'_{i+1}) \in S$ per ogni $1 \leq i \leq k-1$ ed $e'_k = e$

Una lista $L = (E; t; S)$ è detta ordinata se le chiavi contenute nei suoi elementi sono disposte in modo tale da soddisfare una relazione d'ordine totale: per ogni $e_1, e_2 \in S$ allora la chiave di e_1 precede quella di e_2 nella relazione d'ordine totale.

Una lista viene rappresentata come una struttura dati dinamica lineare, in cui ogni elemento contiene solo l'indirizzo dell'elemento successivo (lista singolarmente collegata) oppure anche l'indirizzo dell'elemento precedente (lista doppiamente collegata).

L'indirizzo dell'elemento successivo contenuto nell'ultimo elemento di una lista è indefinito, così come l'indirizzo dell'elemento precedente contenuto nel primo elemento di una lista doppiamente collegata. Fa eccezione il caso dell'implementazione circolare di una lista, nella quale l'ultimo elemento è collegato al primo elemento.

Gli elementi di una lista non sono necessariamente memorizzati in modo consecutivo; quindi, l'accesso ad un qualsiasi elemento avviene scorrendo tutti gli elementi che lo precedono. Questo accesso indiretto necessita dell'indirizzo del primo elemento della lista, detto testa, il quale è indefinito se e solo se la lista è vuota.

Riassumendo:

- La lista è essere una sequenza di elementi di un determinato tipo.
- È un multi-insieme: ci possono essere ripetizioni del medesimo elemento.
- È una sequenza (collezione ordinata): siamo in grado di individuare il primo elemento della sequenza, il secondo, ecc.

Per realizzare una lista è necessaria una rappresentazione collegata, mediante puntatori e allocazione dinamica di memoria.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si memorizzano gli elementi associando a ognuno di essi l'informazione (riferimento) che permette di individuare la locazione in cui è inserito l'elemento successivo (rappresentazione collegata).

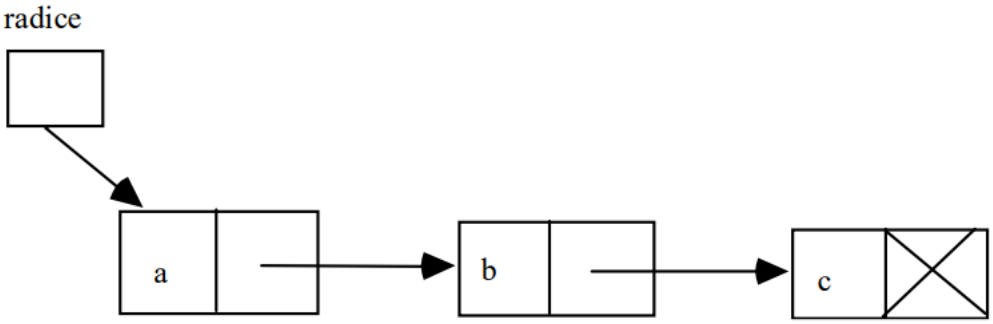


Figura 1 *- Rappresentazione collegata*

Utilizziamo il linguaggio C per implementare la lista mediante puntatori: la memoria utilizzata viene gestita dinamicamente ed è esattamente proporzionale al numero degli elementi della lista.

Ciascun elemento della lista è una struttura di due campi:

- un campo rappresenta il valore dell'elemento (ad esempio un carattere: char)
- un campo è di tipo puntatore e punta all'elemento successivo nella lista (NULL, nel caso dell'ultimo elemento).

```
typedef char element_type;

typedef struct list_element {
element_type value;
      struct list_element *next;
} item;
```

Nella definizione del tipo struct list_element si fa precedere l'identificatore del tipo alla collezione dei campi: in questo modo è possibile dichiarare in tale collezione un campo (next) di tipo puntatore a struct list_element. Inoltre, il tipo struct list_element viene rinominato come item.

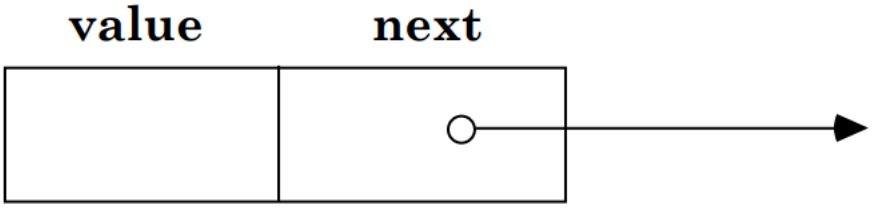


Figura 2 *- Elemento della lista*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per realizzare una lista è necessaria una rappresentazione collegata, mediante puntatori e allocazione dinamica di memoria.

Definiamo dunque:

```
typedef item *list;
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Operazioni sulla lista

La procedura per popolare la lista può essere costruita nel seguente modo:

```
int main() {
  list root=NULL, aux;
  char n;
  while ((n=getchar()) != '\n') {
    aux=(list)malloc(sizeof(struct list_element));
    aux->value=n;
    aux->next=root;
    root=aux;
  }
}
```

Il ciclo while legge i valori dallo standard input, fino a incontrare il carattere di fine file (ritorno a capo); per ogni valore letto viene allocato un nuovo elemento (con l'operazione malloc) il cui indirizzo è assegnato ad aux.

Al campo aux->value viene assegnato il valore n, mentre al campo aux->next è il valore di root.

Il nuovo elemento è collocato in testa alla lista e risulta essere il primo della sequenza. Per questo motivo a root è assegnato l'indirizzo del nuovo elemento che è contenuto in aux. Per il particolare modo di creazione, il campo puntatore dell'ultimo elemento della lista è sempre NULL (importante per verificare la fine della lista)

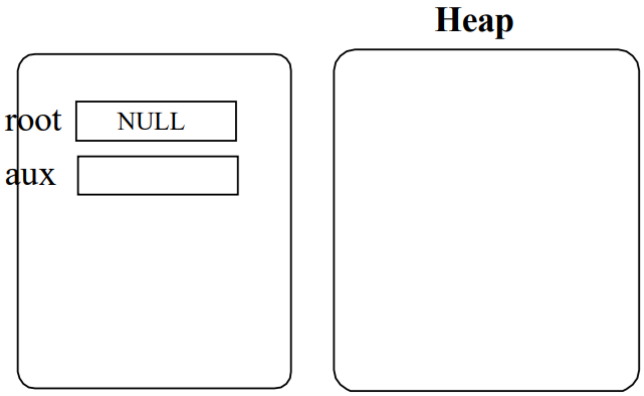
Quindi la lista è creata in ordine inverso rispetto alla sequenza data in ingresso.

Se volessimo rappresentare graficamente cosa accadrebbe nel momento in cui si inserisce in sequenza: a b \n

PEGASO
Università Telematica

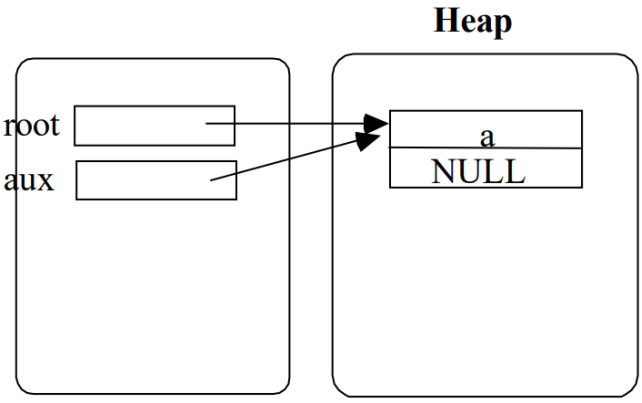
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Inizialmente abbiamo semplicemente root che punta NULL ed aux semplicemente dichiarata:



Quando l'utente inserisce a viene eseguita la seguente porzione di codice:

```
aux=(list)malloc(sizeof(struct list_element));
aux->value=n;
aux->next=root;
root=aux;
```

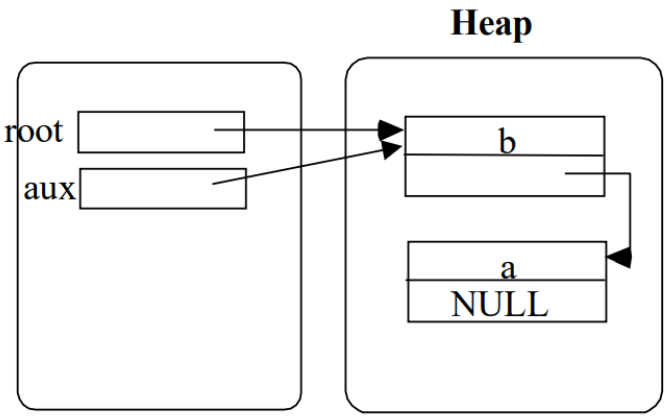


Quando l'utente inserisce b viene eseguita la seguente porzione di codice:

```
aux=(list)malloc(sizeof(struct list_element));
aux->value=n;
aux->next=root;
root=aux;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



E se volessimo memorizzare i valori letti in modo tale da averli nello stesso ordine della sequenza di ingresso? Occorrerebbe inserire ogni nuovo elemento in fondo alla lista.

Per ottenere questo tipo di inserimento serve un puntatore (per esempio, aux) aggiornato via via e che punti all'ultimo elemento della lista (oltre al puntatore root).

Si dovrebbe procedere come nella figura seguente:

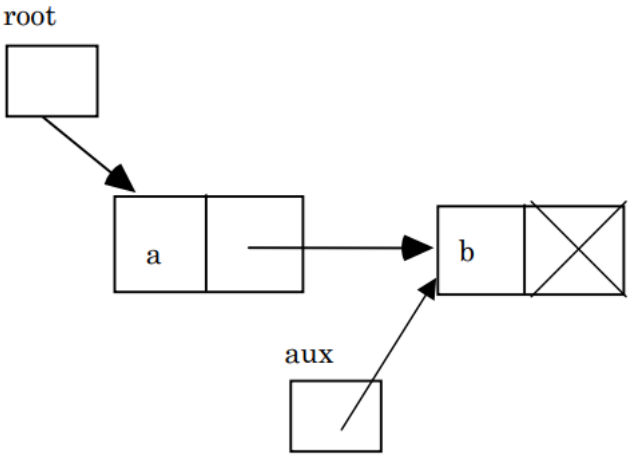


Figura 3 *- Aux punta all'ultimo elemento*

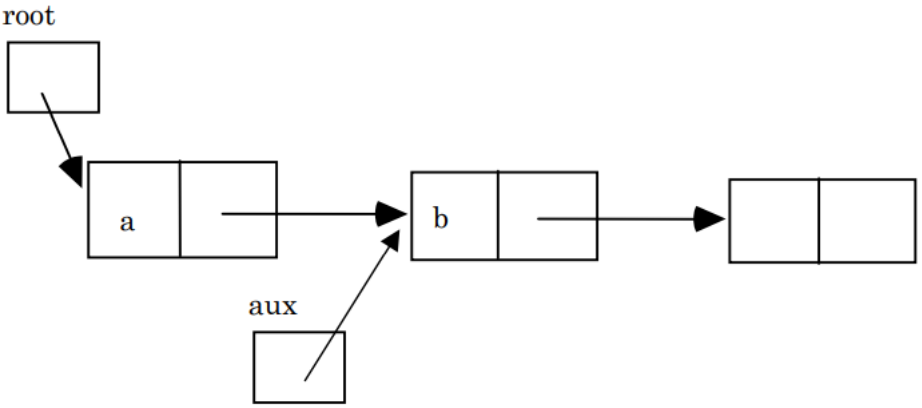


Figura 4 *- Viene inserito un nuovo elemento*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

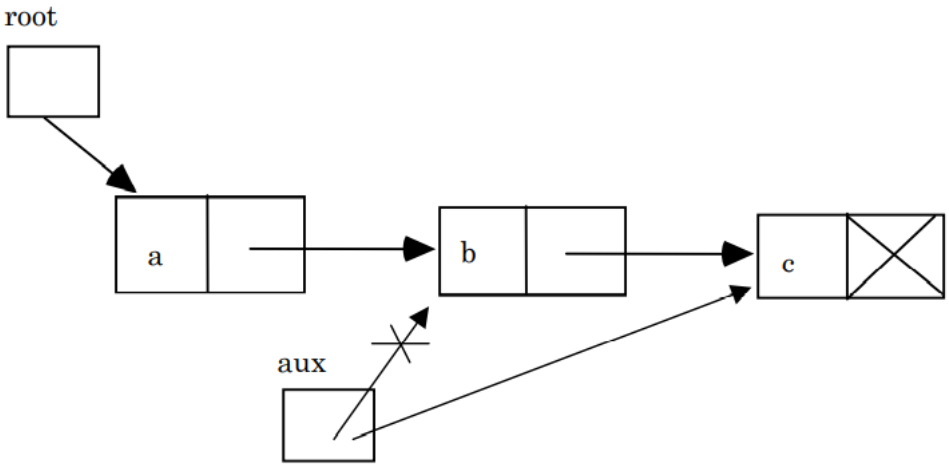


Figura 5 *- Aux viene aggiornato*

Tale procedura può essere costruita nella seguente maniera:

```
void inserisce_dietro() {
  list root=NULL, aux;
  char n;
  while ((n=getchar()) != '\n') {
    if (root==NULL)  {
      root=(list)malloc(sizeof(struct list_element));
      aux=root;
    } else {
      aux->next=(list)malloc(sizeof(struct list_element));
      aux=aux->next;
    }
    aux->value=n;
    aux->next=NULL;
  }
}
```

Indichiamo di seguito anche la funzione per la stampa dei dati presenti all'interno della lista; in realtà è possibile sviluppare tale funzione in modalità iterativa e ricorsiva, pertanto riportiamo entrambe le versioni:

```
// PRINT ITERATIVA
void showlist(list l) {
  printf("[");
  while (l!=NULL) {
    putchar(l->value);
    l=l->next;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
      if (l!=NULL)
        printf(",");
    }
    printf("]\n");
  }

  // PRINT INTERNA RICORSIVA
  void showr_list(list l) {
    if (l!=NULL) {
      putchar(l->value);
      if (l->next!=NULL) {
        printf(",");
        showr_list(l->next);
      }
    }
  }

  // PRINT RICORSIVA
  void showr_l(list l) {
    printf("[");
    showr_list(l);
    printf("]\n");
  }
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Implementazione con smart pointers

A differenza di linguaggi come il **Java** o il GO che possiedono uno heap garbage collector, in C e C++ occorre esplicitamente rilasciare tutta la memoria acquisita dinamicamente ed occorre farlo nel momento giusto. Occorre assolutamente evitare che i riferimenti a memoria precedentemente rilasciata restino accessibili da parti del programma ancora in esecuzione.

Può essere molto complicato e costoso trovare la causa di una **heap corruption** in una applicazione: la difficoltà principale quando si corrompe lo heap è che il processo può proseguire nella sua esecuzione e terminare in maniera anomala anche molto tempo dopo che la corruzione abbia avuto effettivamente luogo.

A complicare le cose, inoltre, resta il fatto che una volta capita la causa, spesso la correzione di un bug di heap corruption mette in discussione l'intero disegno progettuale di una soluzione software.

Gli **smart pointers** rappresentano una evoluzione significativa rispetto ai classici **raw pointers** in stile C: forniscono un modello di controllo della memoria dinamica efficace, senza dover pagare il costo di un garbage collector in esecuzione nel nostro programma.

L'idea vincente di uno smart pointer rispetto al classico raw pointer è l'aggiunta dell'informazione necessaria per capire quando la risorsa può essere rilasciata.

Un raw pointer non possiede nessun'altra informazione se non il valore che denota un indirizzo nello spazio di indirizzamento riservato allo heap. Qualunque scope all'interno del nostro programma che abbia accesso ad un raw pointer a memoria dinamica deve porsi la domanda se, una volta che la abbia utilizzata, debba o meno rilasciare tale memoria.

Gli smart pointer risolvono il problema della deallocazione della memoria ad un costo computazionalmente accettabile e sopratutto quantificabile; aspetto non vero quando si utilizza la garbage collection. In essi è contenuta l'informazione per capire che all'uscita da uno scope, nessun altro scope potrà più accedere alla memoria puntata.

Distinguiamo tra **unique_ptr** e **shared_ptr**:

- **unique_ptr**: è un smart pointer che rappresenta la proprietà esclusiva di un oggetto. Ciò significa che una volta che un oggetto viene assegnato a un unique_ptr, il unique_ptr diventa l'unico proprietario di quell'oggetto e nessun altro smart pointer o raw pointer dovrebbe accedere a quell'oggetto. Quando il unique_ptr viene distrutto, l'oggetto che gestisce verrà automaticamente distrutto.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **shared_ptr**: è un smart pointer che gestisce la proprietà condivisa di un oggetto. Ciò significa che più di un shared_ptr può gestire lo stesso oggetto e l'oggetto verrà distrutto solo quando tutti i shared_ptr che lo gestiscono verranno distrutti. Questo è utile quando più parti del codice devono accedere allo stesso oggetto e il suo ciclo di vita è legato all'ultimo shared_ptr che gestisce l'oggetto.

In generale, è consigliabile utilizzare unique_ptr quando si desidera una proprietà esclusiva dell'oggetto e shared_ptr quando si desidera la proprietà condivisa dell'oggetto (unique_ptr è più leggero e più veloce di shared_ptr a causa della sua implementazione più semplice).

L'operazione di make_* in C++ è una funzione di utilità che consente di creare un'istanza di uno smart pointer con la seguente sintassi:

```
unique_ptr<int> p = make_unique<int>(10);
shared_ptr<int> p = make_shared<int>(10);
```

La funzione make_unique crea un'istanza di un unique_ptr che gestisce un oggetto di tipo specificato. Questo è particolarmente utile perché unique_ptr non ha un costruttore di copia, il che significa che non è possibile creare facilmente una copia di un unique_ptr. Inoltre, make_unique può essere utilizzato per creare unique_ptr che gestiscono array dinamici.

L'operazione di move invece consiste nello spostare la proprietà di un oggetto da uno smart pointer ad un altro. Ciò significa che il primo smart pointer non gestirà più l'oggetto e il secondo smart pointer diventerà il proprietario dell'oggetto; ad esempio, se abbiamo due unique_ptr a e b, possiamo trasferire la proprietà dell'oggetto gestito da a a b con il codice seguente:

```
unique_ptr<int> a = make_unique<int>(10);
unique_ptr<int> b = move(a);
```

È possibile eseguire una implementazione alternativa basata su unique_ptr al fine di non dover gestire la parte di "pulizia" (si può evitare qualsiasi "delete"):

```
// LinkedListUP.h

#ifndef LINKED_LIST_UP
#define LINKED_LIST_UP

#include <memory>

using namespace std;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
class LinkedListUP {
  private:
    struct Node {
    int data;
    shared_ptr<Node> next;
    Node(int data) : data(data), next(nullptr) {}
    Node(int data, shared_ptr<Node> next) : data(data), next(std::move(next)) {}
  };

  shared_ptr<Node> head;

  public:
    LinkedListUP();

    void insertAtStart(int value);
    void insertAtEnd(int value);
    void deleteNode(int value);
    void printList();
};

#endif

#include <iostream>
#include "LinkedListUP.h"

LinkedListUP::LinkedListUP() {
  head=NULL;
}

void LinkedListUP::insertAtStart(int value) {
  auto new_node = make_shared<Node>(value);
  new_node->next = move(head);
  head = move(new_node);
}

void LinkedListUP::insertAtEnd(int value) {
  if (!head) {
      head = make_shared<Node>(value);
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    } else {
        auto current = head.get();
        while (current->next) {
            current = current->next.get();
        }
        current->next = make_shared<Node>(value);
    }
}

void LinkedListUP::deleteNode(int value) {
    if (!head) {
        return;
    }

    if (head->data == value) {
        head = move(head->next);
        return;
    }

    auto current = head.get();
    while (current->next && current->next->data != value) {
        current = current->next.get();
    }

    if (current->next) {
        current->next = move(current->next->next);
    }
}

void LinkedListUP::printList() {
    if (!head){
      cout << "List is empty" << endl;
      return;
    }
    auto curr = head;
    while (curr) {
        cout << curr->data << " ";
        curr = curr->next;
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
        }
        cout<<endl;
    }
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Analisi della complessità

È facile verificare che tali algoritmi hanno una complessità "lineare"; infatti, se la lista ha n elementi e l'elaborazione del valore di ogni elemento si svolge al più in d passi, in ogni caso la complessità è:

$$T(n) = 1 + n \cdot (1 + d + 1) + 1 = (d + 2) \cdot n + 2 = O(n)$$

Similmente, per la ricerca dell'elemento nella lista, qualora l'elemento fosse in prima posizione si avrebbe: $T(n) = O(1)$

Mentre nel caso peggiore si avrebbe una complessità lineare:

$$T(n) = 1 + n \cdot (1 + 1) + 1 = 2 \cdot n + 2 = O(n)$$

Discorso analogo per l'inserimento e la cancellazione:

- Caso ottimo: $T(n) = O(1)$
- Caso peggiore: $T(n) = O(n)$

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley
- Sedgewick: Algoritmi in C, Pearson, 2015
- Cormen Leiserson Rivest Stein-Introduzione Agli Algoritmi E Strutture Dati-Prima Edizione

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).